

Evaluating LLM Limitations in Static Code Analysis Through Complex C Programs

Yousif Al-Jaf
a12413675@univie.ac.at
[GitHub repo](#)

Prof. Dipl.-Ing. Dr. Sebastian Schrittwieser
sebastian.schrittwieser@univie.ac.at

Abstract - This paper investigates the ability of large language models (LLMs) to statically predict the runtime output of intentionally complex C programs. Five C programs were designed using techniques drawn from cryptography and classical computer arithmetic to maximize analysis difficulty. Three prompts of increasing specificity were constructed and tested across four AI model configurations accessed through their web interfaces, ChatGPT 5.5 High, ChatGPT 5.4 Instant, Gemini 3.1 Pro, and Qwen 3.7 Plus, and additionally through an automated pipeline against the open-weight Llama-3.3-70B model via the Groq API. A total of 60 manual evaluation trials plus a full automated run were conducted across all models, programs, and prompt conditions. Results reveal a clear capability split: ChatGPT 5.5 High and Qwen 3.7 Plus correctly predicted output across all five programs and all three prompts, while ChatGPT 5.4 Instant failed on all 15 trials, Gemini 3.1 Pro showed partial success only on specific programs, and Llama-3.3-70B failed on all five programs. Thinking time did not reliably predict accuracy. These findings demonstrate that code complexity techniques rooted in cryptographic primitives provide strong resistance to LLM static analysis, and that model architecture rather than prompt quality is the primary determinant of success.

Index Terms - chain-of-thought reasoning, code complexity, large language models, prompt engineering, static code analysis

INTRODUCTION

Large language models have demonstrated remarkable capability across a wide range of code understanding tasks, including generation, completion, explanation, and bug detection. However, their ability to perform precise static analysis, that is, determining the exact runtime output of a program without executing it, remains poorly characterized, particularly when programs

are designed to resist such analysis through deliberate structural complexity.

This research addresses the following primary question: Can intentionally complex C programs, combined with structured prompt engineering that suppresses chain-of-thought reasoning, reliably prevent LLMs from predicting program output? Understanding the answer to this question has direct relevance to several applied areas, including automated code review, AI-assisted security auditing, and the broader study of LLM reasoning limitations under adversarial input conditions.

To investigate this, five C programs were constructed using techniques including data-dependent function pointer dispatch, mutual recursion with non-obvious termination, linear feedback shift registers, multi-word carry chain arithmetic, and lockstep cross-dependent state machines. A preprocessing and submission pipeline was developed in Bash to strip comments, shuffle function order, and submit preprocessed code to a hosted LLM API. Three prompts of increasing constraint were evaluated across four AI models, producing 60 total manual evaluation trials and additionally through an automated run against the Groq API. All results were compared against the program's original output obtained by compiling and executing each program under GCC with the C99 standard.

What originally motivated this work was a simple but nagging question: if AI models are now confidently used to read and explain code, how well do they actually perform when the code is genuinely hard to trace? The programs in this study were not picked randomly. Each one was built around a specific technique known to resist analysis, and testing them against real models with real prompts felt like the most honest way to find out where the limits actually are.

PROGRAM DESIGN

Five C programs were constructed, each employing a distinct primary complexity technique. All programs were compiled with gcc using the C99 standard and O0 optimization flag on a little-endian x86-64 system. All arithmetic is unsigned and wrapping.

Program 1 (c01) Function Pointer Dispatch: This program stores eight different mathematical transformation functions in an array and uses the current value being processed to decide which function to call at each step, meaning the execution path changes based on the data itself rather than a fixed sequence. At each of 12 steps, an index is computed by XORing the current 32-bit state with a version of itself shifted right by 4 bits, then masking to 3 bits to produce a number between 0 and 7. This index selects a function from the pointer table, each applying a different combination of multiplication, XOR, bitwise complement, and rotation, and the state is replaced with the result. Because the index at every step depends entirely on the output of the previous step, no function call in the chain can be determined without computing all prior steps exactly, making the full dispatch path opaque to static analysis.

Program 2 (c02) Mutual Recursion: Two functions call each other back and forth like a telephone chain, each transforming the value slightly before passing it to the other, and the chain stops at different points depending on different conditions in each function. Alpha computes a new value using multiplication and XOR with a depth counter, then calls beta; beta applies a left rotation and an additive term scaled by depth, then calls alpha back. Alpha terminates when the lower 4 bits of the current value equal 0xA, while beta terminates when they equal 0x5. Because both functions use different arithmetic and the lower 4 bits evolve unpredictably through multiply and rotate operations, neither termination point can be anticipated without tracing every recursive call. The final output is the XOR of two separately traced chains, both of which must be completed before the answer is known.

Program 3 (c03) Linear Feedback Shift Register: This program implements the same technique used in old game console random number generators and stream ciphers, where a sequence of bit shifts and XOR operations produces a stream of values that looks statistically random despite being entirely deterministic. At each step, the LFSR reads the bits at positions 31, 21, 1, and 0 of the current 32-bit state, XORs them together to produce a single new bit, shifts the entire state right by one position, and places the new bit at position 31. The tap positions 31, 21, 1, and 0 are specifically chosen to produce a maximal-length sequence, meaning the register

cycles through 2 to the power of 32 minus 1 different states before repeating. Three separate chains are run for 17, 31, and 13 steps, respectively, on differently derived seeds, and all three results are XORed together. After even a small number of steps, every output bit is a function of many input bits through a long cascade of XOR and shift operations, and there is no analytical shortcut.

Program 4 (c04) Carry Chain Arithmetic: The program performs addition across four linked 32-bit numbers treated as one large 128-bit value, where overflow from one slot carries into the next just like carrying a digit in long addition by hand, then scrambles all four values by XORing them with their neighbors and rotating their bits. Two arrays of four 32-bit words are constructed from the seed using different derivations, and multi-word addition is performed using a 64-bit accumulator to capture the carry out of each 32-bit slot into the next. After the first addition pass, each word is XORed with its neighbor in the array and rotated left by 7 bits; then the full addition is performed a second time with carry propagation again across all four words. A carry bit generated by word 0 overflowing changes word 1, which changes the XOR-rotate result for word 1, which changes whether word 1 generates a carry into word 2, creating a cascade of dependencies across all four words through both passes.

Program 5 (c05) Lockstep State Machines: Two counters run side by side for 10 steps, where each counter's new value depends not only on itself but on the other counter as well, meaning neither can be computed independently and both must be tracked simultaneously throughout all steps. The first state variable s1 is updated by rotating itself left 3 bits and XORing with s2 multiplied by the constant 0x1D, while the second state variable s2 is updated by rotating itself right 5 bits and adding the result of s1 XORed with 0xA3B2. Critically, both new values are computed from the same current pair before either one is updated, so the update is simultaneous rather than sequential. After 10 such steps, the final output is the XOR of both states trimmed to 8 bits. A single wrong bit in either machine at any step corrupts both machines for all remaining steps, since each feeds into the other continuously.

PRE-PROCESSING PIPELINE

To make the evaluation fair and consistent, a Bash shell script was written to handle the preprocessing before anything gets sent to the Groq API. It does three things: strips out all the comments so the model gets no hints, shuffles the function order so it cannot read top to bottom naturally, and adds a short preamble describing the compiler environment. The cleaned-up file is then sent to

the API, and whatever the model predicts is saved for comparison against the actual compiled output.

PROMPT CONSTRUCTION

Three prompts of increasing specificity were constructed and tested independently across all four models and all five programs.

Prompt 1 (Baseline): A minimal prompt asking the model to state what the program prints when compiled and run. No constraints on reasoning style or output format.

Prompt 2 (Structured Trace): The model is instructed to trace every function call step by step, track the exact value of every variable at every step, and output only results it can confirm by tracing. Guessing is explicitly prohibited.

Prompt 3 (Full Environment Constraint): The model is given the exact compilation environment including gcc, O0, C99, little-endian architecture, sizeof(int)=4, wrapping arithmetic, and logical right shifts on unsigned types. It is required to trace the full program per seed, tracking exact uint32_t values at every step, and explicitly instructed to declare uncertainty rather than approximate if it loses track of a value.

PROGRAM OUTPUTS

The correct outputs for the primary test program are as follows:

```
c01_fptr_dispatch.c
dispatch(0011) = 0x1A
dispatch(00AB) = 0xFA
dispatch(0200) = 0xBD
dispatch(FFFF) = 0x8D
```

```
c02_mutual_recursion.c
run(0033) = 0xF5
run(00CC) = 0x40
run(01FF) = 0x10
run(BEEF) = 0x37
```

```
c03_lfsr.c
chain(ACE1) = 0x1E
chain(1234) = 0x34
chain(DEAD) = 0x52
chain(0001) = 0xFE
```

```
c04_carry_chain.c
crunch(0042) = 0x03
crunch(1337) = 0x81
crunch(ABCD) = 0xB4
crunch(FF00) = 0x12
```

```
c05_state_machines.c
lockstep(0001) = 0x12
lockstep(0080) = 0xBA
```

```
lockstep(1234) = 0x21
lockstep(FFFF) = 0x30
```

MODELS EVALUATED

Four model configurations were evaluated: ChatGPT 5.5 High (reasoning-enabled flagship), ChatGPT 5.4 Instant (fast inference variant), Gemini 3.1 Pro (with both standard and extended thinking modes), and Qwen 3.7 Plus (extended reasoning chain model). All trials used a temperature of 0, where available, to ensure deterministic outputs.

EVALUATION PROTOCOL

Each program was compiled and executed to obtain the program’s original correct output. Each model was then queried with each of the three prompts applied to the preprocessed source. For the Groq API evaluation, a Bash shell script automated the preprocessing and submission pipeline, stripping comments, shuffling function order, and injecting a noise preamble before sending each file to the API and recording the model response. For the manual evaluations conducted against ChatGPT 5.5 High, ChatGPT 5.4 Instant, Gemini 3.1 Pro, and Qwen 3.7 Plus, each C source file and its corresponding prompt were submitted manually through each model’s graphical user interface in a private chat session. A new private chat was opened for each combination of file and prompt to ensure no context carried over between trials, preventing any model from drawing on prior submissions when generating its response. Responses were scored as correct (all four seed outputs matching the compiled program’s output), partial (some outputs correct), or incorrect (all outputs wrong). Response time was recorded from prompt submission to final output. A total of 60 evaluation trials were conducted: 4 models x 5 programs x 3 prompts.

RESULTS

A. ChatGPT 5.5 High

ChatGPT 5.5 High achieved correct output on all five programs across all three prompt conditions, making it the only model to achieve a perfect score of 15/15 trials. Response times ranged from 6 to 48 seconds. Notably, more complex prompts generally required more thinking time, but accuracy was maintained throughout. Full results are shown in Table I.

TABLE I: ChatGPT 5.5 High Results.

Program	Prompt 1	P1 Time	Prompt 2	P2 Time	Prompt 3	P3 Time
c01_fptr_dispatch.c	Correct	6s	Correct	6s	Correct	6s
c02_mutual_recursion.c	Correct	12s	Correct	12s	Correct	12s
c03_lfsr.c	Correct	18s	Correct	18s	Correct	18s
c04_carry_chain.c	Correct	24s	Correct	24s	Correct	24s
c05_state_machines.c	Correct	48s	Correct	48s	Correct	48s

c01	Correct	7s	Correct	34s	Correct	27s
c02	Correct	6s	Correct	27s	Correct	40s
c03	Correct	7s	Correct	38s	Correct	48s
c04	Correct	9s	Correct	22s	Correct	29s
c05	Correct	6s	Correct	26s	Correct	36s

B. ChatGPT 5.4 Instant

ChatGPT 5.4 Instant failed on all five programs across all three prompt conditions, scoring 0/15 trials. Despite spending up to 63 seconds on some trials under Prompt 3, no correct output was produced for any program or seed. Increased prompt specificity produced no improvement whatsoever. Full results are shown in Table II.

TABLE II: ChatGPT 5.4 Instant Results.

Program	Prompt 1	P1 Time	Prompt 2	P2 Time	Prompt 3	P3 Time
c01	Incorrect	9s	Incorrect	15s	Incorrect	60s
c02	Incorrect	8s	Incorrect	10s	Incorrect	50s
c03	Incorrect	5s	Incorrect	16s	Incorrect	25s
c04	Incorrect	8s	Incorrect	40s	Incorrect	50s
c05	Incorrect	7s	Incorrect	54s	Incorrect	63s

C. Gemini 3.1 Pro

Gemini 3.1 Pro produced the most varied results of any tested model. It failed completely on c01 and c02 across all prompt conditions despite spending 3 to 6 minutes reasoning. On c03 with extended thinking, it correctly predicted output for all three prompts, though Prompt 3 produced only a partial result, with the fourth output incorrect. On c04, it achieved partial correctness under all three prompts, correctly predicting 2 of 4 outputs in each case. On c05, it produced no correct output despite spending up to 4 minutes. Full results are shown in Table III.

TABLE III: Gemini 3.1 Pro Results.

Program	Prompt 1	P1 Time	Prompt 2	P2 Time	Prompt 3	P3 Time
c01	Incorrect	~4 min	Incorrect	~5 min	Incorrect	~6 min
c02	Incorrect	~3 min	Incorrect	~4 min	Incorrect	~5 min
c03	Correct	~3.5 min	Correct	~4 min	Partial (3/4)	~5 min
c04	Partial (2/4)	~3.5 min	Partial (2/4)	~4.5 min	Partial (2/4)	~2.5 min
c05	Incorrect	~1.5 min	Incorrect	~2 min	Incorrect	~4 min

D. Qwen 3.7 Plus

Qwen 3.7 Plus achieved correct output on all five programs across all three prompt conditions, matching ChatGPT 5.5 High with a perfect score of 15/15 trials. Response times ranged from 27 seconds to 2 minutes. An interesting finding is that for c05, response time actually decreased as prompt complexity increased, from 1 minute under Prompt 1 to 43 seconds under Prompt 3, suggesting the additional constraints helped the model focus its reasoning more efficiently. Full results are shown in Table IV.

TABLE IV: Qwen 3.7 Plus Results.

Program	Prompt 1	P1 Time	Prompt 2	P2 Time	Prompt 3	P3 Time
c01	Correct	40s	Correct	1 min	Correct	2 min
c02	Correct	30s	Correct	1.3 min	Correct	1.8 min
c03	Correct	27s	Correct	43s	Correct	1.2 min
c04	Correct	39s	Correct	48s	Correct	56s
c05	Correct	1 min	Correct	45s	Correct	43s

E. Llama-3.3-70B via Groq API

To validate the automated preprocessing pipeline and extend the evaluation to an open-weight model, the

open-source Llama-3.3-70B-Versatile model was tested through the Groq API. Each program was submitted via the Bash script described in the pre-processing pipeline, which stripped comments, shuffled function order, injected the noise preamble, and requested only the final stdout with no explanation.

Llama-3.3-70B failed on all five programs, producing no correct output for any seed. The most revealing failure occurred on c02, where the model returned the identical value 0x4E for all four distinct seed inputs, a textbook example of blind guessing rather than tracing. It also misread the input BEEF as 0BEEF, indicating it was not carefully parsing the source. Table V presents the model's predicted output against each of the program's outputs.

TABLE V: Llama-3.3-70B via Groq API results.

Program	Seed	Original program output	Llama Result Output	Result
c01	0011	0x1A	0x4E	✗
	00AB	0xFA	0x55	✗
	0200	0xBD	0x8F	✗
	FFFF	0x8D	0x4B	✗
c02	0033	0xF5	0x4E	✗
	00CC	0x40	0x4E	✗
	01FF	0x10	0x4E	✗
	BE EF	0x37	0x4E	✗
c03	ACE1	0x1E	0x4A	✗
	1234	0x34	0x4B	✗
	DEAD	0x52	0x03	✗
	0001	0xFE	0x34	✗
c04	0042	0x03	0x3E	✗

	1337	0x81	0x55	✗
	ABCD	0xB4	0x1D	✗
	FF00	0x12	0x55	✗
c05	0001	0x12	0x6E	✗
	0080	0xBA	0x45	✗
	1234	0x21	0x55	✗
	FFF0	0x30	0x7A	✗

OVERALL SCORE SUMMARY

Table VI summarizes the overall accuracy score for each model.

TABLE VI: Overall Model Performance Summary.

Model	Score	Access	Failure type
ChatGPT 5.5 High	15/15 (100%)	Manual	None
Qwen 3.7 Plus	15/15 (100%)	Manual	None
Gemini 3.1 Pro	4/15 (27%)	Manual	Partial + Incorrect
ChatGPT 5.4 Inst.	0/15 (0%)	Manual	Blind Guessing
Llama-3.3-70B	0/5 (0%)	Groq API	Blind Guessing

ANALYSIS AND DISCUSSION

A. Clear Model Capability Split

The results reveal a sharp binary split in model capability for this class of tasks. ChatGPT 5.5 High and Qwen 3.7 Plus both achieved perfect scores of 15/15, succeeding on every program under every prompt condition. ChatGPT 5.4 Instant and Llama-3.3-70B failed on all trials without exception. Gemini 3.1 Pro fell in between, succeeding on specific programs but failing broadly. This suggests that the ability to statically trace complex C programs is not a gradual capability that scales smoothly with model size, but rather a threshold capability that either a model has or does not have for a given complexity level.

B. Thinking Time Does Not Predict Accuracy

One of the most significant findings is that thinking time had no reliable relationship with accuracy. ChatGPT 5.4 Instant spent up to 63 seconds on c05 under Prompt 3 and produced incorrect output. Gemini 3.1 Pro spent up to 6 minutes on c01 across multiple prompts and was consistently wrong. By contrast, ChatGPT 5.5 High solved c01 correctly in 7 seconds under Prompt 1 and c05 correctly in 6 seconds. Qwen 3.7 Plus solved c03 correctly in 27 seconds. These results demonstrate that the bottleneck is not reasoning time but the model's ability to maintain an exact 32-bit integer state across chains of data-dependent operations without accumulating error.

C. Prompt Complexity Effect

Prompt specificity had a measurable but model-dependent effect on response time, with minimal effect on accuracy for capable models. For ChatGPT 5.5 High, average response time increased from 7 seconds under Prompt 1 to 36 seconds under Prompt 3, while accuracy remained at 100%. For Qwen 3.7 Plus, response time also generally increased with prompt complexity, though c05 showed the reverse pattern. For ChatGPT 5.4 Instant and Llama-3.3-70B, increased prompt complexity produced longer or unchanged response times but no improvement in accuracy, confirming that prompt engineering alone cannot compensate for insufficient model capability on this task class.

D. Gemini Partial Success Analysis

Gemini 3.1 Pro's partial success pattern is analytically interesting. It succeeded fully on c03 (LFSR) under standard prompt conditions but failed on c01 (dispatch) and c05 (lockstep) even with extended thinking enabled. This suggests Gemini may have a specific strength in sequential bit-manipulation tasks like LFSR steps but struggles with data-dependent branching and cross-dependent parallel state. The partial correctness on c04 (2 out of 4 outputs correct across all prompts) suggests carry chain arithmetic is within reach for some seeds but not others, likely depending on whether carries propagate in predictable ways for specific input values.

What was genuinely unexpected was that Gemini spent the most time thinking out of all tested models and still got the most wrong, which challenges the intuition that more reasoning time means better results.

E. The Blind Guessing Signature

The automated Groq run against Llama-3.3-70B produced the clearest single piece of evidence for blind guessing in the entire study. On c02, the model returned 0x4E for all four different seed inputs. If the model were

tracing the program, different inputs would almost certainly produce different outputs; returning one constant across four distinct seeds is only explicable as the model emitting a plausible-looking byte without performing the computation. The parallel misreading of the input BEEF as 0BEEF reinforces that the model was pattern-matching on surface features rather than parsing and executing the source.

F. Failure Mode Taxonomy

Three distinct failure modes were identified across all models and prompt conditions.

Failure Mode 1 Blind Guessing: the model produces output quickly without step-by-step reasoning, and all predicted values are incorrect with no structural relationship to the correct output. Observed consistently in ChatGPT 5.4 Instant and Llama-3.3-70B.

Failure Mode 2 Extended Incorrect Reasoning: the model spends significant time reasoning but accumulates integer state errors and produces incorrect results. Observed in Gemini 3.1 Pro on c01, c02, and c05.

Failure Mode 3 Partial Accuracy: the model correctly traces some seeds but fails on others. Observed in Gemini 3.1 Pro on c03 (Prompt 3) and c04 (all prompts), suggesting that difficulty varies per seed within the same program depending on specific bit patterns.

CONCLUSION

This paper has presented a systematic empirical investigation into LLM static code analysis capability across five intentionally complex C programs, three prompt conditions, and five model configurations, totaling 60 manual evaluation trials plus a full automated run. The primary findings are as follows.

First, a clear capability split exists between models. ChatGPT 5.5 High and Qwen 3.7 Plus achieved perfect scores of 15/15, while ChatGPT 5.4 Instant scored 0/15, Gemini 3.1 Pro scored 4/15 with partial successes concentrated on specific programs, and Llama-3.3-70B scored 0/5 on the automated run.

Second, thinking time does not predict accuracy. Models that spent the most time reasoning (Gemini 3.1 Pro, up to 6 minutes) were outperformed on accuracy by models that answered in seconds (ChatGPT 5.5 High, 7 seconds on c01). The bottleneck is the model's ability to maintain an exact integer state, not the time available for reasoning.

Third, prompt quality improves response structure but does not rescue models below the capability threshold. ChatGPT 5.4 Instant failed identically under all three prompt conditions despite spending significantly more time under Prompt 3.

Fourth, partial accuracy patterns reveal program-specific difficulty variation. Gemini's success on c03 but failure on c01 and c05 suggests that specific complexity techniques impose qualitatively different demands on model reasoning.

Fifth, the complexity techniques rooted in cryptographic primitives, LFSR feedback, carry chain arithmetic, and cross-dependent state evolution proved most resistant, consistent with their design as computationally irreducible operations that must be executed step by step rather than reasoned about analytically.

Future work will investigate a broader range of open-weight models through the automated Groq pipeline, expand the number of seed values per program to quantify success rates more precisely, and develop a formal LLM Analysis Resistance Score metric to predict model failure before testing.

REFERENCES

- [1] S. Schrittwieser, S. Katzenbeisser, J. Kinder, G. Merzdovnik, and E. Weippl, "Protecting software through obfuscation: Can it keep pace with progress in code analysis?" *ACM Computing Surveys*, vol. 49, no. 1, pp. 1–37, 2016. doi: 10.1145/2886012
- [2] B. De Sutter, S. Schrittwieser, B. Coppens, and P. Kochberger, "Evaluation methodologies in software protection research," *Computers and Security*, vol. 138, 2024. doi: 10.1016/j.cose.2023.103662
- [3] S. Schrittwieser, S. Katzenbeisser, P. Kieseberg, M. Huber, M. Leithner, M. Mulazzani, and E. Weippl, "Covert computation: Hiding code in code for obfuscation purposes," in *Proc. 8th ACM SIGSAC Symposium on Information, Computer and Communications Security (ASIACCS)*, pp. 529–534, 2013.
- [4] S. Schrittwieser, S. Katzenbeisser, P. Kieseberg, M. Huber, M. Leithner, M. Mulazzani, and E. Weippl, "Covert computation: Hiding code in code through compile-time obfuscation," *Computers and Security*, vol. 42, pp. 13–26, 2014.
- [5] L. Kummer, S. Moustafa, S. Schrittwieser, W. N. Gansterer, and N. M. Kriege, "Attacking graph neural networks with bit flips: Weisfeiler and Leman go indifferent," in *Proc. 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 1428–1439, Aug. 2024.
- [6] M. Chen et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.
- [7] J. Austin et al., "Program synthesis with large language models," arXiv preprint arXiv:2108.07732, 2021.
- [8] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022.